PERSONAL PROJECT - ARCHITECTURE PLAN

Points Tracker: Maximize Value for a Specific Trip

Unified balances across Chase, hotels, and airlines - with a trip-level optimizer that answers "what should I pay with, and when should I transfer?"

Stack: Python (FastAPI) + Postgres + React + Claude API + AwardWallet API. **Primary user:** Stanley. **Core job:** track every balance you hold, detect changes and expirations, and recommend the cheapest points-or-cash path for one trip at a time.

The product in one paragraph

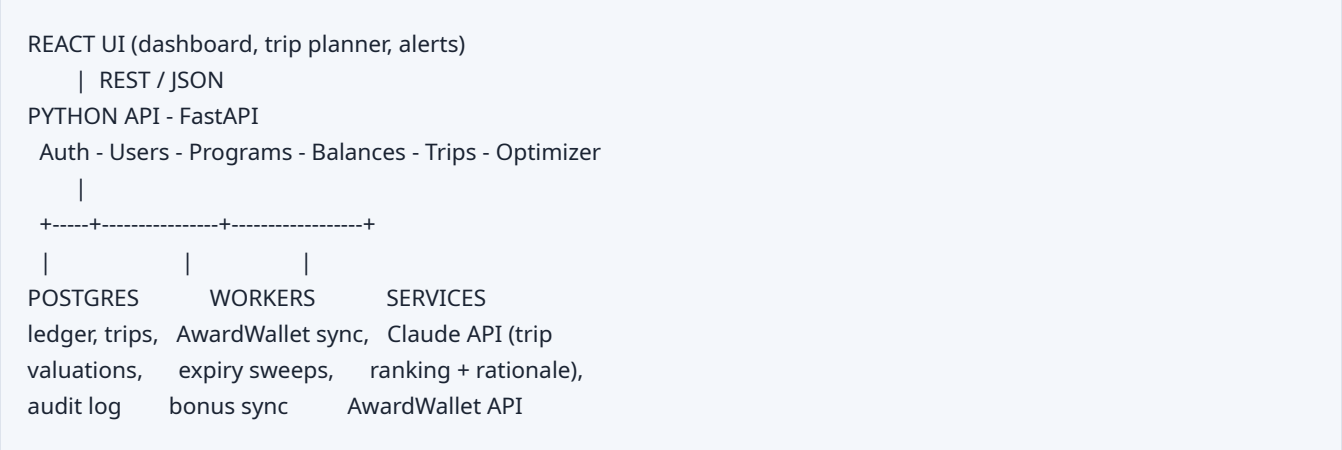
Most points tools track one program. Your value leaks across programs: Chase Ultimate Rewards sitting idle while a Hyatt stay could absorb them at 2+ cents per point, or airline miles expiring while a transfer bonus is active. This app treats all balances as one portfolio. Balances sync from AwardWallet in Phase 1 - no email parsing, no bank passwords in this app. You enter a trip - dates, destination, cabin, hotel nights - and deterministic code enumerates every payment option, scored in dollars saved and cents per point (cpp) realized. Claude then ranks those computed options and explains the trade-offs. The result: pay cash, burn existing hotel/airline balances, or transfer Chase points (with bonus applied) - with the math shown, not guessed.

> **Design principle:** never overwrite a balance. Every earn, burn, transfer, and expiration is a ledger event. That single decision makes historical valuation, anomaly alerts ("was this transfer yours?"), and undo possible - and it is what separates this from a spreadsheet.

Prepared October 2, 2026. Programs referenced for design: Chase Ultimate Rewards, World of Hyatt, plus major airline and hotel transfer partners. Transfer partners and ratios change; the schema stores them as data, not code.

System architecture: three tiers, one ledger

Keep the shape boring and proven: a React frontend, a Python API, a Postgres ledger, and background workers. Claude sits behind the API as a ranking service, so a model change never touches the ledger - and Claude never computes balances or prices; deterministic code does that first.



Components

Component	Tech	Responsibility
Web app	React + Vite + TypeScript	Balances dashboard, trip planner, alert inbox, settings
API	Python 3.12, FastAPI, SQLAlchemy	Auth, CRUD, valuation engine, deterministic optimizer, Claude orchestration
Database	Postgres 16	Ledger events, balance snapshots, program catalog, valuations, trips, alerts
Workers	Python + Celery or RQ + Redis	AwardWallet sync, expiry sweeps, transfer-bonus sync
Balance source	AwardWallet Account Access API	Balance snapshots and account activity history across programs
AI	Claude API	Ranks pre-computed trip options and explains them; no ledger writes, no price or balance invention

Why this shape

- **Postgres is the product.** Balances, history, and trip math must be exact and auditable; Postgres with row-level constraints beats any document store here.
- **Workers keep the API fast.** AwardWallet syncs and Claude calls are slow; they never block a page load.
- **AI behind an interface, math in code.** Claude ranks and explains; enumeration, cpp, and dollars-saved are computed deterministically from the ledger first.

Authentication and credential storage

Auth is deliberately small in Phase 1: a single-user tool needs no built-in identity system, but its few secrets are named explicitly. The browser holds almost nothing, Postgres never holds a usable password, and this app never stores bank or loyalty-program passwords - AwardWallet holds those (see the tradeoff below).

How you log in to the app

- **Phase 1 (single user, recommended):** do not build a login system. Run the app behind your home network or Tailscale, or put HTTP Basic Auth on the reverse proxy.
- **If it becomes multi-user:** store only `argon2id` password hashes in a `users` table - never plaintext - and use server-side sessions referenced by an `HttpOnly`, `Secure`, `SameSite` cookie.
- **What not to do:** never store JSON Web Tokens (JWTs) in browser `localStorage`; any cross-site scripting (XSS) injection can exfiltrate them. The session cookie is the safer default.

Where each credential lives

Credential	Stored where	Protection
Your app login (if multi-user)	`users` table in Postgres	`argon2id` hash only; session ID in an `HttpOnly` cookie
AwardWallet OAuth tokens	`credentials` / `secrets` table in Postgres	Encrypted at rest with AES-GCM / Fernet; key in an environment variable, never in the repo or database. Request only the scope needed (`read_balances`, plus activity read if used)
AwardWallet API key	Environment variables / secret manager	IP-restricted to the app server; never in frontend code, never committed
Claude API key	Environment variables / secret manager	Server-side only; the browser never calls Claude directly
Bank / loyalty-program passwords	Not in this app - held by AwardWallet only	Never collected by this app and never screen-scraped. You authorize AwardWallet once; this app receives only balances and activity via its API

> **The credentials that matter:** the AwardWallet OAuth tokens and API key - whoever holds them can read every balance AwardWallet shares. They get encryption at rest, with keys outside the database. **The tradeoff, plainly:** current balances require someone to log in to Chase, Southwest, Hyatt, and the rest. AwardWallet does that for you, so your program credentials live at AwardWallet, not in this app. That is a conscious trust decision - manual entry and CSV import remain if you would rather not delegate it.

How balance tracking works without stored logins in this app

This app polls no bank directly. AwardWallet refreshes accounts on its side and exposes balances and activity via its Account Access API; the sync worker writes those into the ledger. If it disconnects, manual / CSV entry keeps the ledger usable.

Data model: an event-sourced ledger

Balances are derived, never stored as a mutable number. The ledger is the source of truth.

Core tables

Table	Key columns	Notes
`programs`	id, name, type (bank / hotel / airline), currency_name	Chase UR, World of Hyatt, United MileagePlus, etc.
`accounts`	id, user_id, program_id, member_ref (last 4), status	One row per loyalty account you hold
`ledger_events`	id, account_id, kind (earn / burn / transfer_in / transfer_out / expire / adjust / snapshot), points, occurred_at, source, external_ref	Append-only. external_ref dedupes AwardWallet activity (AwardWallet transaction / account IDs); source is awardwallet, manual, or csv
`balance_snapshots`	id, account_id, points, captured_at, source, external_ref	AwardWallet (or manual) point-in-time balances; reconciled against the ledger - a mismatch raises an alert
`transfer_partners`	from_program_id, to_program_id, ratio_num, ratio_den, active, bonus_pct, bonus_ends_at	Ratios and bonuses as data; update without deploys
`valuations`	program_id, cpp_cents, effective_date, source	Your baseline value per program; time-versioned
`trips` / `trip_legs`	trip: origin, destination, dates, travelers; leg: kind (flight / hotel), cabin, nights, cash_price_cents	The optimizer's input
`redemption_options`	trip_leg_id, program_id, points_cost, cash_copay_cents, cpp_realized, rank	Optimizer output, recomputed on demand
`alerts`	id, user_id, kind, payload, seen_at	Unusual transfer, expiration within 90 days, bonus match

Balance read path

A materialized view sums `ledger_events` per account for the ledger-derived balance. AwardWallet snapshots land in `balance_snapshots` on every sync, so the dashboard shows both: derived vs. last-synced. When they disagree beyond a small tolerance, the app flags it rather than overwriting - the snapshot is evidence, the ledger is the audit trail. Dated snapshots also power the "balance over time" chart.

> **Expiration handling:** programs differ - some expire on inactivity, some never. Store `expires_at` per account plus its rule, and sweep daily for alerts at 90 / 30 / 7 days. Never silently zero a balance.

Ingestion: how balances stay current

There is no email ingestion in this design. Balances come from one primary source - AwardWallet - with manual entry and CSV import as fallback. Every source writes the same ledger events and snapshots, so the rest of the app never knows where a number came from.

Phase 1 - AwardWallet sync (primary, ships first)

- **Connect once:** register a free AwardWallet business / developer account, get an API key (IP-restricted to the app server), and authorize access with OAuth2. Request only the granular access needed - `read_balances` for balances, plus activity/history read if you want transaction detail.
- **Client:** use the AwardWallet Python client on PyPI (`pip install awardwallet`) from the sync worker, or call the Account Access API directly - same contract either way.
- **What it supplies:** current balance snapshots, expiration dates, elite status, and account activity history for the programs you have linked in AwardWallet (Chase, Southwest, Hyatt, United, and 460+ others) - without this app ever seeing a program password.
- **How it writes:** each sync writes a dated row to `balance_snapshots` and converts new activity into `ledger_events`, deduped on `external_ref` (the AwardWallet account / transaction identifier). Re-running a sync is idempotent.
- **Cadence and limits:** this is near-current, not literally real-time. AwardWallet refreshes periodically and on demand (minutes to hours). Free accounts limit manual refreshes per account per day; trigger a sync on dashboard load (throttled), on a schedule, and on demand before running the trip optimizer.

Fallback - Manual entry and CSV import (also Phase 1)

- Quick-add form: program, points, date, note. Covers any program AwardWallet cannot link, and opening balances on day one.
- CSV import for historical backfill and for activity you download yourself while logged in to a program. Imports land in a review queue before they hit the ledger.
- Screenshot check: a monthly "is this balance still right?" prompt reconciles drift for accounts with thin activity data.

What is deliberately not built

- **No AI email parsing and no Gmail ingestion.** An earlier draft parsed transfer-confirmation emails with an LLM; it is removed. AwardWallet activity is already structured - parsing unstructured email to reconstruct it adds cost, error, and a Gmail OAuth surface for no benefit.
- **No direct bank / airline APIs and no screen-scraping.** Chase, Southwest, Hyatt, and peers offer no public consumer API for point balances. This app does not log in to them; AwardWallet does, on its side of the API.

Transfer partner and bonus sync

A small curated table (`transfer_partners`) updated manually or from a weekly check. When a bonus is active (for example, a percentage uplift on transfers to a hotel or airline partner), the optimizer uses the effective ratio automatically and labels the option "bonus ends [date]."

Valuation and the trip optimizer

This is the feature that makes the tracker worth opening. Everything before it exists to make its inputs trustworthy.

Valuation model

- Store a baseline cents-per-point per program in `valuations` (your numbers, editable in settings; sensible defaults to start).
- Every redemption option computes **cpp realized** = (cash price - cash copay) / points cost. Options below the program baseline are flagged "poor value."
- Portfolio view shows total points and an estimated dollar value per program and overall - clearly labeled an estimate.

Trip optimizer flow - deterministic math, Claude ranking

The optimizer is a hybrid by design. Code does the arithmetic; Claude does the judgment call. Neither does the other's job.

```
1. You create a trip: destination, dates, travelers,
   flight legs and hotel nights, with cash prices.
2. DETERMINISTIC ENGINE enumerates payment options per leg:
   - pay cash
   - burn an existing hotel/airline balance
   - transfer Chase points (ratio + active bonus applied)
   - points + cash copays where the program offers them
3. Each option scored IN CODE: dollars saved, cpp realized,
   points remaining after, expiration risk.
4. Backend sends Claude ONE structured request: the computed
   options + current balances, transfer ratios/bonuses, and
   expiration dates. No raw ledger dump, no open-ended prompt.
5. Claude returns a RANKED RECOMMENDATION as structured JSON:
   ordered option IDs, a rationale per option, trade-offs, and
   warnings - validated against the option set before display.
```

Rules the optimizer must respect

- **Guardrail:** Claude ranks and explains options computed from real ledger data - it does not invent prices, balances, or live award availability. If Claude's response references an option, price, or balance that was not in the request payload, the backend rejects it and falls back to the code-computed ranking.
- Never recommend a transfer you cannot reverse - label all Chase transfers "irreversible" in the UI.
- Never recommend spending points that expire sooner on a later trip when an earlier use exists; surface the trade-off instead - this is exactly the kind of nuance Claude's rationale should call out.
- Any option based on typical award costs must be labeled "verify availability before transferring." Live availability is not checked in v1; the label is mandatory, in both the API response and the UI.
- Cash is always an option. Sometimes cash wins; the plan must be willing to say that.

Claude: trip redemption optimization

The one AI job: ranking a specific trip's options

AI has exactly one job in this app: given a specific trip, judge which of the already-computed redemption options is best, and explain why in plain language. The backend - never the browser - sends requests to the Claude API. There is no email parsing and no general chatbot.

Stage	Who does it	Detail
Enumerate options	Deterministic code	Per trip leg: cash, burn a balance, transfer Chase points (ratio + bonus), points + cash. Inputs come only from the ledger, `transfer_partners`, and the trip's cash prices
Score options	Deterministic code	Dollars saved, cpp realized, points remaining after, expiration risk - all computed, none estimated by the model
Rank + explain	Claude API	Receives the scored options plus balances, transfer ratios / bonuses, and expiration data as structured context; returns structured JSON: ranked option IDs, rationale, trade-offs, warnings
Validate + render	Deterministic code	Response is schema-validated and checked against the option set; unknown IDs or invented figures are rejected and the code ranking is shown instead

Guardrail: Claude ranks and explains options computed from real ledger data - it does not invent prices, balances, or live award availability. Options based on typical award costs are labeled "verify availability before transferring" in Claude's output requirements, the API response, and the UI. Claude never writes to the ledger.

Claude request design (concrete)

- **Trigger:** `POST /trips/{id}/optimize` - optionally triggers a fresh AwardWallet sync first, so the ranking uses the latest balances.
- **Payload:** trip legs with cash prices; per-leg option list with points cost, cash copay, dollars saved, and cpp; current balances with `expires_at`; active transfer ratios and bonuses with end dates; your per-program cpp baselines.
- **Output contract:** structured JSON only (enforced with a response schema) - `ranked_options[]` (option ID, rank, rationale, warnings), plus a short overall summary. No free-form numbers: every figure Claude cites must already exist in the payload.
- **Cost control:** one Claude call per optimize run, options pre-filtered in code (drop clearly dominated options first); cache the result on the trip until balances or prices change.

API surface and frontend

API surface (FastAPI)

Endpoint	Purpose
`GET /balances`	Current balance per account (ledger-derived + last AwardWallet snapshot) + portfolio value
`GET /ledger?account_id=`	Event history with source and external refs
`POST /events`	Manual earn / burn / transfer entry (fallback path)
`POST /awardwallet/connect` / `GET /awardwallet/status`	Start the AwardWallet OAuth flow; show link status, scopes, and last sync time
`POST /awardwallet/sync`	Trigger a sync now; writes snapshots + deduped activity events, returns a sync diff
`POST /imports/csv` / `POST /imports/review`	Upload a CSV; approve or reject queued import rows
`POST /trips` / `GET /trips/{id}/options`	Create a trip; get the code-computed, scored options per leg (no Claude call)
`POST /trips/{id}/optimize`	Run the hybrid optimizer: compute options, call Claude, return the validated ranked plan
`GET /alerts`	Expirations, snapshot-vs-ledger mismatches, bonus matches

Frontend screens (React)

- **Dashboard:** balance cards per program with last-synced time, portfolio value, balance-over-time chart, expiring-soon strip.
- **Ledger:** filterable event table; every row shows its source (AwardWallet, manual, CSV).
- **Trip planner:** trip form, scored options per leg, Claude's ranked plan with rationale and warnings, transfer simulator ("transfer 20,000 Chase points with a 30% bonus = 26,000 partner points"). Typical-cost options carry the "verify availability before transferring" label.
- **Alerts inbox:** one bundled daily view - snapshot mismatches, expirations, and bonus matches, each with a one-line action.
- **Settings:** per-program cpp baselines, AwardWallet connection and sync status, notification thresholds.

Build order: usable in phase 1

Phase	Ships	Done when
1 - Ledger + AwardWallet	Programs, accounts, ledger, dashboard, history chart; AwardWallet connect (OAuth2, `read_balances`) with scheduled + on-demand sync writing snapshots and deduped activity; manual / CSV fallback	Real Chase, hotel, and airline balances appear in one dashboard from AwardWallet - and a re-sync is idempotent
2 - Claude trip optimizer	Trips and cash prices; deterministic enumeration (transfer math, bonuses, dollars saved, cpp); Claude ranking endpoint returning validated structured JSON; availability labeling	For one real trip, the ranked plan beats your manual math and cites only computed figures
3 - Freshness and alerts	Expiration sweeps, snapshot-vs-ledger reconciliation, bonus sync, daily bundled alerts	Expiring points and mismatches surface before they cost you
4 - Live availability (optional)	Award-search integration where an official source exists	Options distinguish "typical cost - verify availability" from "verified available"

Suggested repo layout

```
points-tracker/
  api/       FastAPI app, models, valuation, optimizer, claude_client
  workers/   AwardWallet sync, sweeps, bonus sync
  web/       React app (dashboard, planner, alerts)
  migrations/   Alembic
  docker-compose.yml   Postgres + Redis + api + web
```

Decisions to make before coding

- Single-user (you) or multi-user from day one? Single-user keeps Phase 1 small; the schema already carries `user_id` if that changes.
- AwardWallet trust call + scopes: linking programs there means it stores those logins - Phase 1 proceeds on that basis (unlinked programs stay manual / CSV), starting with `read_balances` only.
- Which programs first? Chase Ultimate Rewards, World of Hyatt, and the two airlines you fly most. Add programs as data rows, not code. Set your starting cpp baselines at the same time - the optimizer's flags are only as good as these numbers.
- Claude failure behavior: on failure or failed validation, show the code-computed ranking - never block the plan on the model.

> **First build step:** stand up Postgres (`programs` , `accounts` , `ledger_events` , `balance_snapshots`); connect AwardWallet and run the first sync; enter anything it cannot link as opening `adjust` events; ship the dashboard.